

Internship Report – Week 6, Task 4

Task Title: Develop a Custom Script to Automate a Common Security Task

Task Type: Python-based Automation Script for Log Analysis

1. Objective

The objective of this task was to create a **custom Python script** that automates the process of analyzing Apache web server logs to detect potential malicious activities such as:

- SQL Injection attempts
- Cross-Site Scripting (XSS) payloads
- Directory Traversal attacks
- Command Injection attempts

The goal was to save security analysts time by automating repetitive log scanning, thereby improving incident response efficiency.

2. Tools & Environment

- **Operating System:** Kali Linux 2025.2
 - Linux virus 6.12.33+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.33-1kali1 (2025-06-25) x86_64 GNU/Linux
 - **Web Server:** Apache 2.4.x
 - **Programming Language:** Python 3.x
 - **Log Source:** /var/log/apache2/access.log
-

3. Implementation Steps

1. **Task Selection:** Decided to automate Apache log analysis since the server already had Apache & ModSecurity in previous tasks.
2. **Pattern Definition:** Defined regex patterns for common web attacks.
3. **Script Development:**
 - Opened the log file in read mode.
 - Searched for suspicious patterns.

- Displayed alerts with log line numbers.

4. Testing:

- Injected simulated malicious entries into access.log.
- Verified that the script correctly flagged them.

5. Refinement:

- Added error handling for missing files.
- Made script executable for quick use.

4. Script Code

```
#!/usr/bin/env python3

import re, os

LOG_FILE = "/var/log/apache2/access.log"

patterns = {
    "SQL Injection": r"(\bUNION\b|\bSELECT\b|\bDROP\b|\bINSERT\b)",
    "XSS": r"<script>|%3Cscript%3E",
    "Directory Traversal": r"\\.\\.\\|\\%2e\\%2e/",
    "Command Injection": r"(\bcat\b|\bwget\b|\bcurl\b).*"
}

def analyze_logs():
    if not os.path.exists(LOG_FILE):
        print(f"[!] Log file not found: {LOG_FILE}")
        return
    with open(LOG_FILE, "r", errors="ignore") as f:
        lines = f.readlines()
    print(f"\n[+] Analyzing {LOG_FILE}...\n")
```

```
for i, line in enumerate(lines, start=1):  
    for attack_type, pattern in patterns.items():  
        if re.search(pattern, line, re.IGNORECASE):  
            print(f"[ALERT] {attack_type} detected in line {i}:")  
            print(f" {line.strip()}\n")  
  
if __name__ == "__main__":  
    analyze_logs()
```

5. Sample Test Output

Tested with simulated log entries:

192.168.0.5 - - [12/Aug/2025:15:32:10 +0500] "GET /index.php?id=1 UNION SELECT password FROM users HTTP/1.1" 200 1234

192.168.0.6 - - [12/Aug/2025:15:35:22 +0500] "GET /search?q=<script>alert(1)</script> HTTP/1.1" 200 4321

Script Output:

[+] Analyzing /var/log/apache2/access.log...

[ALERT] SQL Injection detected in line 201:

192.168.0.5 - - [12/Aug/2025:15:32:10 +0500] "GET /index.php?id=1 UNION SELECT password FROM users HTTP/1.1" 200 1234

[ALERT] XSS detected in line 202:

192.168.0.6 - - [12/Aug/2025:15:35:22 +0500] "GET /search?q=<script>alert(1)</script> HTTP/1.1" 200 4321

6. Findings

- The script was able to **quickly identify attack attempts** in real-time logs.
- Automation reduced the **manual log review time** from hours to seconds.

- False positives can occur if benign requests contain certain keywords — future improvements could involve contextual analysis.
-

7. Recommendations

- Integrate the script with **cron jobs** for scheduled automated scans.
 - Extend detection patterns to cover more attack types.
 - Send alerts via email or Slack to the security team for immediate action.
 - Store suspicious logs in a separate file for forensic investigation.
-

8. Conclusion

This task successfully demonstrated the development of a **practical, reusable security automation tool**. The Python log analyzer increased efficiency and provided a foundation for more advanced intrusion detection mechanisms.
